

ECE253/CSE208 Introduction to Information Theory

Lecture 9: Lossless Source Coding

Dr. Yu Zhang

ECE Department

University of California, Santa Cruz

- Chap 5 and 13 of *Elements of Information Theory (2nd Edition)* by Thomas Cover & Joy Thomas

Data Compression

Taxonomy of Data Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Source coding: Mapping from a sequence of symbols from an information source to a sequence of alphabet symbols (usually bits) such that the source symbols can be

- exactly recovered from the binary bits: **lossless source coding**.
- recovered within some distortion: **lossy source coding**.

We now study six schemes of lossless coding:

1. Huffman coding
2. Shannon-Fano coding
3. Shannon-Fano-Elias coding
4. Shannon coding
5. Arithmetic coding
6. Lempel-Ziv coding



Taxonomy of Data Compression Methods

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

	Category 1	Category 2
Length	Fixed-length e.g., ASCII, fixed block coding	Variable-length e.g., Huffman, Shannon, Arithmetic coding
Model	Known PMF (model-based) e.g., Shannon code, Huffman code	Universal (model-free) e.g., LZ77, LZ78, CTW
Structure	Tree-based e.g., Huffman tree, Shannon-Fano	Interval-based e.g., Arithmetic coding
Coding Unit	Symbol-by-symbol e.g., Huffman coding	Block / Sequence e.g., Arithmetic coding, Lempel-Ziv
Optimality	Minimize expected length	Minimax / Competitive
Distortion	Lossless e.g., PNG, ZIP, FLAC	Lossy e.g., JPEG, MP3

David Huffman

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

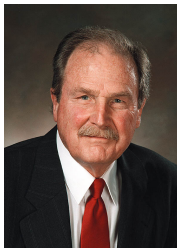


Figure: David A. Huffman (August 9, 1925 – October 7, 1999)

- In 1951, David Huffman (then a PhD student at MIT) developed Huffman coding for a class taught by Robert Fano, improving upon Shannon–Fano coding.
- He later joined UC Santa Cruz in 1967, co-founded its Computer Science Department, served as chair from 1970 to 1973, and retired in 1994.

Huffman Coding is a Workhorse

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding



- Huffman coding remains a core component of practical compression systems, including GZIP, PKZIP, JPEG, PNG, and MPEG.
- Arithmetic coding can achieve better compression, but its adoption was historically limited by patent restrictions.

Huffman Coding Algorithm

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Goal: Construct an optimal prefix-free code minimizing the average code length $\bar{L}$.

Construction

1. **Initialize:** Treat each symbol probability p_i as a leaf node.
2. **Merge:** Repeatedly combine the two nodes with the smallest probabilities. Replace them by a new internal node with weight equal to their sum.
3. **Terminate:** Continue until a single root node remains.
4. **Label:** Assign 0 and 1 to each branch. The codeword for each symbol is the path from the root to its leaf.

Decoding

Start at the root. Read bits sequentially: 0 $\rightarrow$ go left; 1 $\rightarrow$ go right.

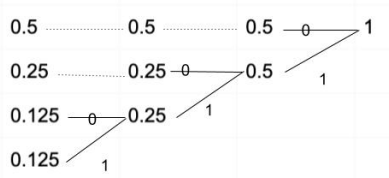
When a leaf is reached, output the corresponding symbol.

Notes: the construction algorithm only requires repeatedly selecting the two smallest nodes. Efficient implementation uses a *priority queue* with complexity $\mathcal{O}(n \log n)$.

Huffman Coding: Example 1

Consider 4 symbols with $\Pr(A) = 0.5$, $\Pr(B) = 0.25$, $\Pr(C) = 0.125$, $\Pr(D) = 0.125$.

Symbol	Code	Code length	Probability
A	0	1	0.5
B	10	2	0.25
C	110	3	0.125
D	111	3	0.125



- Average code length: $\bar{L} = \sum_i p_i l_i = 1.75 = H(X)$, which achieves the lower bound with no redundancy.
- If we use a fixed length code, the average code length is 2.

Huffman Coding: Example 2

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

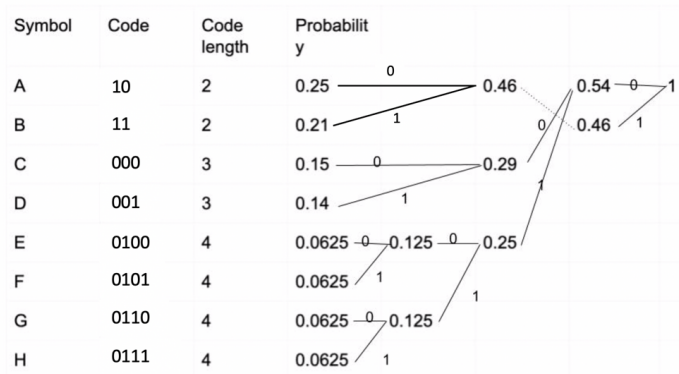
Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Consider the following 8 symbols. We have $\bar{L} = 2.79 > 2.78 = H(X)$.



Block Huffman Coding

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Consider 2 symbols with $\Pr(A) = 0.9, \Pr(B) = 0.1$. So, $\bar{L} = 1$ and $H = 0.47$ bits per symbol. Combine two symbols: AA, AB, BA, BB, we get:

Symbol	Code	Code length	Probability			
AA	0	1	0.81	0.81	0.81	0
AB	100	2	0.09	0.1	0.19	1
BA	11	3	0.09	0.09		
BB	101	3	0.01			

$$\bar{L} = \frac{1}{2} \times (1 \times 0.81 + 2 \times 0.09 + 3 \times 0.09 + 3 \times 0.01) = 0.645 \text{ bits/symbol.}$$

Block Huffman coding reduces the average code length per symbol.

Dummy Symbols in Huffman Tree

Huffman code is prefix free, because it can always be represented by a D -ary tree.

Codeword	X	Probability
1	1	0.25
2	2	0.25
01	3	0.2
02	4	0.1
000	5	0.1
001	6	0.1
002	Dummy	0.0

Figure: Dummy symbols may be needed to construct the D -ary tree (for $D \geq 3$).

- Each merge reduces the number of nodes by $(D - 1)$. After k merges, only one node remains.
- Thus,

$$k \times (D - 1) + 1 = N + E,$$

where N is the number of source symbols and E is the number of dummy symbols.

- Hence the necessary condition for a full D -ary tree:

$$N + E \equiv 1 \pmod{D - 1}.$$

Structure of an Optimal Prefix Code

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Lemma

For any distribution, there exists an optimal prefix code (minimizing expected length) that satisfies:

- **Monotonicity of lengths:**

$$p_j > p_i \Rightarrow l_j \leq l_i.$$

More probable symbols are assigned no longer codewords.

- **Sibling property at maximum depth:** The two longest codewords have the same length, correspond to the two least probable symbols, and differ only in the last bit.
- Not every optimal code satisfies the second property, but one can always rearrange an optimal code to obtain this structure.

These structural properties enable the greedy merging step in Huffman's algorithm.

Optimality of Huffman Code

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Theorem

Among all uniquely decodable codes, the Huffman code achieves minimum average length:

$$\bar{L}_H \leq \bar{L}_{UD}$$

for every uniquely decodable code with the same source distribution.

Interpretation

Huffman coding is optimal for *symbol-by-symbol* compression when the source distribution is known.

Optimality of Huffman Code: Proof

Proof by induction on m (number of symbols). Assume $p_1 \geq p_2 \geq \dots \geq p_{m-1} \geq p_m$ and $l_1 \leq l_2 \leq \dots \leq l_{m-1} = l_m$.

- *Induction hypothesis:* Huffman is optimal for $m - 1$ symbols (trivial for $m = 2$).
- In an optimal code, p_{m-1} and p_m are siblings with equal length. Merge them into a single symbol with probability

$$p' = p_{m-1} + p_m.$$

- Let C_m be the Huffman code for m symbols and C_{m-1} the Huffman code after merging. Then

$$\bar{L}(C_{m-1}) = \bar{L}(C_m) - (p_{m-1} + p_m).$$

- For any optimal code C'_m , merging the two least probable symbols yields

$$\bar{L}(C'_{m-1}) = \bar{L}(C'_m) - (p_{m-1} + p_m).$$

- By the induction hypothesis,

$$\bar{L}(C_{m-1}) \leq \bar{L}(C'_{m-1}) \implies \bar{L}(C_m) \leq \bar{L}(C'_m).$$

Redundancy of Huffman Code

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

The redundancy of the Huffman code of a source with PMF p satisfies:

$$0 \leq L_H - H(p) \leq 1$$

The gap goes to 1 when $H(p) \rightarrow 0$.

Question: Typically how large is it?

Theorem (Gallager 1978')

$$L_H - H(p) \leq p_{\max} + c,$$

where $p_{\max}$ is the probability of the most likely symbol, and $c = \log(\frac{2}{e} \times \log e) \approx 0.086$.

Furthermore, if $p_{\max} \geq 0.5 \implies L_H - H(p) \leq 2 - H(p_{\max}) - p_{\max} \leq p_{\max}$.

Shannon-Fano (SF) Coding Algorithm

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

- S1): Sort the given source symbols according to probability, put the most frequent symbols on the left.
- S2): Divide the sorted list into two parts (always keep the order of the symbols, **NO shuffle**), whose total probabilities should be as close as possible to being equal.
- S3): Code the left part as 0, while the right part as 1. Keep doing until each symbol becomes a leaf code.
- S4): Read off the code strings from the root to all leaf nodes, which yields the SF codes.

Shannon-Fano (SF) Coding: Example (1/2)

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

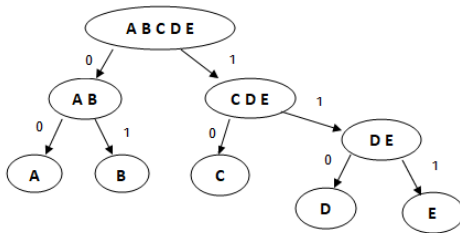
Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Symbol	A	B	C	D	E
Count	15	7	6	6	5
Probability	0.385	0.179	0.154	0.154	0.128
Huffman code	1	000	001	010	011
SF code	00	01	10	110	111



Shannon-Fano (SF) Coding: Example (2/2)

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

We compute the average code length per symbol as

$$L_{\text{SF}} = \frac{2 \times (15 + 7 + 6) + 3 \times (6 + 5)}{39} = 2.28 \text{ bits/sym}$$

$$L_{\text{H}} = \frac{1 \times 15 + 3 \times (7 + 6 + 6 + 5)}{39} = 2.23 \text{ bits/sym}$$

$$H(X) = - \sum_i p_i \log(p_i) = 2.185 \text{ bits/sym}$$

$$L_{\text{SF}} > L_{\text{H}} > H(X)$$

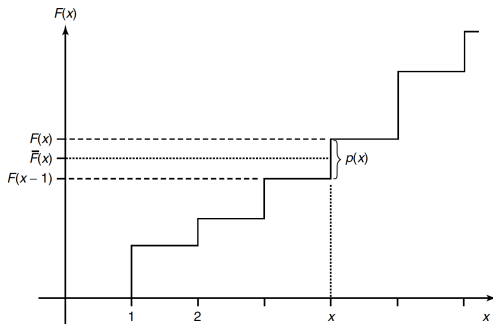
Insight. Shannon-Fano coding is a *top-down* approach in constructing the tree, while Huffman coding is *bottom-up*.

Shannon-Fano-Elias (SFE) Coding

SFE coding (an interval method) uses the midpoint CDF

$$\overline{F}(x) = \sum_{y < x} p(y) + \frac{1}{2}p(x),$$

instead of the PMF.



SFE Coding Algorithm

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Assume all source symbols have been sorted in a certain sense (e.g., lexicographic).

S1): Let $Z(x)$ be the binary representation of $\overline{F}(x)$ (using a decimal-to-binary converter).

S2): Choose the code length $l(x) = \lceil \log \frac{1}{p(x)} \rceil + 1$.

S3): The encode string $C(x)$ is the first $l(x)$ most significant bits (MSB) after the decimal point of $Z(x)$.

SFE Coding: Example

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Consider $X = \{A, B, C, D\}$ with $p_i = \{\frac{1}{3}, \frac{1}{4}, \frac{1}{6}, \frac{1}{4}\}$

- $\overline{F}(A) = \frac{1}{2}P(A) = (0.1666)_{10} \Rightarrow Z(A) = (0.00101010)_2$

$$l(A) = \lceil \log \frac{1}{p(A)} \rceil + 1 = 3 \Rightarrow A \rightarrow \mathbf{001}$$

- $\overline{F}(B) = P(A) + \frac{1}{2}P(B) = (0.458333)_{10} \Rightarrow Z(B) = (0.011101010)_2$

$$l(B) = \lceil \log \frac{1}{p(B)} \rceil + 1 = 3 \Rightarrow B \rightarrow \mathbf{011}$$

- $\overline{F}(C) = (0.6666)_{10} \Rightarrow Z(C) = (0.10101010)_2 \Rightarrow C \rightarrow 1010$

- $\overline{F}(D) = (0.875)_{10} \Rightarrow Z(D) = (0.111)_2 \Rightarrow D \rightarrow 111$

SFE code length

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Since $a \leq \lceil a \rceil < a + 1$, the redundancy is at most 2 bits per symbol:

$$H(X) + 1 \leq L_{SFE} = \sum_{x \in \mathcal{X}} p(x) \times \left(\left\lceil \log \frac{1}{p(x)} \right\rceil + 1 \right) < H(X) + 2.$$

SFE Code is Uniquely Decodable

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Lemma

SFE code is uniquely decodable.

Proof: Let $\lfloor \bar{F}(x) \rfloor_{l(x)}$ be the truncation of $\bar{F}$ to $l(x)$ of MSB after the decimal point. Hence, by the definition of truncation and code length $l(x)$, we have

$$\bar{F}(x) - \lfloor \bar{F}(x) \rfloor_{l(x)} < 2^{-l(x)} \leq 2^{\log p(x)-1} = \bar{F}(x) - F(x-1) \Rightarrow$$

$$F(x-1) < \bar{F}(x) - 2^{-l(x)} < \lfloor \bar{F}(x) \rfloor_{l(x)} \leq \bar{F}(x).$$

Therefore, the codeword $\lfloor \bar{F}(x) \rfloor_{l(x)}$ lies within the step corresponding to x so that we can uniquely identify the symbol x .

SFE Code Is Prefix-Free

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Theorem

SFE code is prefix-free.

Proof.

Let the SFE codeword for x be the first $l(x)$ bits of

$$\overline{F}(x), \quad \text{i.e., } c(x) = 0.z_1z_2 \cdots z_{l(x)}.$$

Then $c(x)$ represents the interval

$$I_x = [c(x), c(x) + 2^{-l(x)}).$$

By construction,

$$I_x \subset [F(x-1), F(x)).$$

Since the intervals $[F(x-1), F(x))$ for different symbols are disjoint, the intervals I_x are also disjoint. Hence, no codeword can be a prefix of another, and the SFE code is prefix-free.

Shannon Coding

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

The Shannon code is defined in a similar way as the SFE code with the exception of

- The symbols are ordered by decreasing probability $p_1 \geq p_2 \geq \dots \geq p_m$
- The codeword of i -th symbol is formed by the $\left\lceil \log \frac{1}{p_i} \right\rceil$ bits of $\tilde{F}_i = \sum_{k=1}^{i-1} p_k$

Shannon code length. Since $l_i = \left\lceil \log \frac{1}{p_i} \right\rceil \implies$

$$\log \frac{1}{p_i} \leq l_i \leq 1 + \log \frac{1}{p_i} \implies H(X) \leq L_S \leq H(X) + 1.$$

Shannon Coding: Example

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

i	p_i	l_i	$\sum_{n=0}^{i-1} p_n$	Previous value in binary	Codeword for a_i
1	0.36	2	0.0	0.0000	00
2	0.18	3	0.36	0.0101...	010
3	0.18	3	0.54	0.1000...	100
4	0.12	4	0.72	0.1011...	1011
5	0.09	4	0.84	0.1101...	1101
6	0.07	4	0.93	0.1110...	1110

Figure: An example of Shannon coding.

Competitive Optimality (1/2)

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Theorem (Shannon code vs any other uniquely decodable code)

Let $l(x)$ be the Shannon code length, and $l'(x)$ that of any other uniquely decodable code. Then

$$\Pr(l(X) \geq l'(X) + c) \leq \frac{1}{2^{c-1}}.$$

Proof:

$$\begin{aligned}\Pr(l(X) > l'(X) + c) &\leq \Pr(-\log_2 p(X) + 1 \geq l'(X) + c) = \Pr(p(X) \leq 2^{-l'(X)-c+1}) \\ &= \sum_x p(x) \times \mathbb{1}(p(x) \leq 2^{-l'(x)-c+1}) \\ &\leq \sum_x 2^{-l'(x)-c+1} \times \mathbb{1}(p(x) \leq 2^{-l'(x)-c+1}) \\ &\leq 2^{1-c} \times \sum_{x \in \mathcal{X}} 2^{-l'(x)} \leq \frac{1}{2^{c-1}} \quad [\text{by McMillan's inequality}]\end{aligned}$$

Competitive Optimality (2/2)

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Theorem

For a dyadic PMF $p(x)$, let $l(x) = \log \frac{1}{p(x)}$ be the codeword lengths of the binary Shannon code. Let $l'(x)$ be the lengths of any other uniquely decodable binary code. Then

$$\Pr(l(X) \leq l'(X)) \geq \Pr(l(X) > l'(X)) ,$$

with equality iff $l'(x) = l(x)$ for all x . The code length $l(x) = \log \frac{1}{p(x)}$ is uniquely competitively optimal.

To this end, we have:

$$H(X) \leq L_H \leq L_S \leq H(X) + 1 \leq L_{SFE} < H(X) + 2.$$

Comparing Different Source Coding

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

- Shannon code takes a probability-matching approach: the code length of a symbol matches the probability of the symbol.
- Fano code uses equal partitioning based on probability: minimize the difference of the probabilities of the two partitioned sets.
- Huffman code uses sorting and merging of symbol sets based on probability: merge the two sets with the least probabilities.

Arithmetic Coding (AC)

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Arithmetic coding extends SFE by encoding an entire message into a single number $q \in [0, 1)$, representing the sequence as a sub-interval of the unit interval rather than coding symbols individually.

Problem with Huffman coding.

- Consider two symbols with: $\Pr(A) = 0.9999, \Pr(B) = 0.0001 \Rightarrow H = 1.47 \times 10^{-3}$.
- If we were to send “A” 1000 times, we might hope to use $1000 \times H = 1.47$ bits. Actually, we require 1000 bits for the Huffman codes.
- We could alleviate this loss by using the block Huffman coding with a codebook of size $|\mathcal{X}|^n$ (need to enumerate all sequences of length n), which increases exponentially with the block length n , **not practical!**
- For certain high-speed applications, the space complexity and speed of Huffman coding is a bottleneck.

AC Algorithm

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Basic ideas.

- The entire message is represented by an interval in $[0, 1)$
- The interval is successively divided into sub-intervals proportional to symbol probabilities
- Final interval size = product of symbol probabilities
- Final code = any value from the interval
- Intervals for distinct message sequences are disjoint
- Decoding computes the same sequence of intervals

AC Algorithm: Example

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

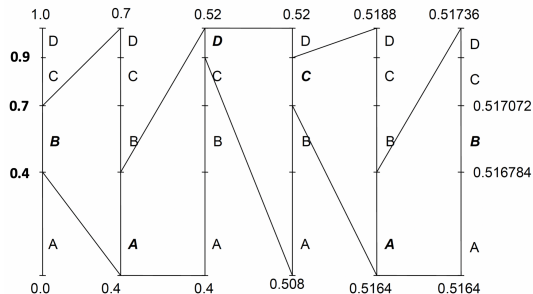


Figure: Sequence "BADCAB" maps to any number $q \in [0.516784, 0.517072]$.

- Encoding with rescaled intervals: each symbol narrows the interval by a factor of p_i .
- Final *sequence interval* size: $s_n = \prod_{i=1}^n p_i$.
- Specifying any number in the final interval uniquely determines the sequence.

AC Algorithm (cont'd)

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Recursive formulas. Consider a message sequence of length n to be encoded by the arithmetic coding. For $i = 1, 2, \dots, n$, let ℓ_i and s_i denote the **low end** and the **size** of the interval after we read off the i -th symbol in the message sequence. The final interval $[\ell_n, \ell_n + s_n)$ can be determined by the following recursions:

$$\boxed{\ell_i = \ell_{i-1} + s_{i-1}f_{k(i)}, \quad s_i = s_{i-1}p_i,} \quad \text{with } \ell_0 = 0, s_0 = 1.$$

- p_i is the probability of the i -th symbol in the sequence, which corresponds to the $k(i)$ -th symbol that has been sorted in the original source set.
- $f_{k(i)} = \sum_{j=1}^{k(i)-1} \tilde{p}_j$ is the modified CDF of the $k(i)$ -th symbol, which sums up the probabilities ($\tilde{p}_j$'s) of all its preceding symbols in the source set.

AC Algorithm (cont'd)

For the above example, the message sequence to be encoded is “BAD CAB”. The sorted source symbols and associated quantities are summarized in the following table:

Sorted symbols	A	B	C	D
Probability	0.4	0.3	0.2	0.1
Modified CDF	0	0.4	0.7	0.9

Use the recursions, we have:

$$\ell_1 = \ell_0 + s_0 f_B = 0 + 1 \times 0.4 = 0.4, \quad s_1 = s_0 p_1 = 1 \times 0.3 = 0.3$$

$$\ell_2 = \ell_1 + s_1 f_A = 0.4 + 0.3 \times 0 = 0.4, \quad s_2 = s_1 p_2 = 0.3 \times 0.4 = 0.12$$

$$\ell_3 = \ell_2 + s_2 f_D = 0.4 + 0.12 \times 0.9 = 0.508, \quad s_3 = s_2 p_3 = 0.12 \times 0.1 = 0.012$$

$$\ell_4 = \ell_3 + s_3 f_C = 0.508 + 0.012 \times 0.7 = 0.5164, \quad s_4 = s_3 p_4 = 0.012 \times 0.2 = 0.0024$$

RealArith Encoding & Decoding

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

- RealArithEncoder:
 1. Determine the low end ℓ_n and the size s_n of the final interval by using the recursions.
 2. Code using the mid-point $m_n = \ell_n + \frac{s_n}{2}$ truncated to $\left\lceil \log \frac{1}{s_n} \right\rceil + 1$ bits.
- RealArithDecoder:
 1. Read bits as needed so code interval falls within a message interval.
 2. Repeat until the entire message sequence of length n has been decoded.

Theorem

Let s_n denote the final probability interval of the RealArithEncoder. The binary representation of $m_n = \ell_n + \frac{s_n}{2}$ ($\text{mid} = \frac{\text{upper} + \text{lower}}{2}$) truncated to $l(x) = 1 + \left\lceil \log \frac{1}{s_n} \right\rceil$ bits is a uniquely decodable code for message x among prefix-free messages.

RealArith Encoding & Decoding (cont'd)

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Theorem (Bound of AC length)

*For a message sequence of length n with self information $\left\{Sl_i := \log \frac{1}{p_i}\right\}_{i=1}^n$,
`RealArithEncode` generates at most $2 + \sum_{i=1}^n Sl_i$ bits.*

Proof:

$$1 + \left\lceil \log \frac{1}{s_n} \right\rceil = 1 + \left\lceil \log \left(\prod_i p_i \right)^{-1} \right\rceil = 1 + \left\lceil \sum_i \log \frac{1}{p_i} \right\rceil < 2 + \sum_{i=1}^n Sl_i.$$

Optimality of AC Coding

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Optimality (expected length of an n -symbol message).

$$\begin{aligned}\bar{L}^{(n)} &= \sum_{x^n \in \mathcal{X}^n} p(x^n) l(x^n) = \sum p(x^n) \left(\left\lceil \log \frac{1}{p(x^n)} \right\rceil + 1 \right) \\ &\leq \sum p(x^n) \left(\log \frac{1}{p(x^n)} + 2 \right) = H(X^n) + 2 \implies\end{aligned}$$

Bits per symbol: $\frac{H(X^n)}{n} \leq L_{AC} \leq \frac{H(X^n)}{n} + \frac{2}{n} \Rightarrow L_{AC} \rightarrow H(\mathcal{X})$ as $n \rightarrow \infty$.

Note: Although this is not necessarily optimal for any fixed block length (Huffman code has a lower average codeword length), the procedure is incremental and can be used for any block length.

Universal Source Coding

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

- Huffman coding and arithmetic coding give at least as good, and often better compression than any universal codes.
- Universal codes are useful when Huffman coding is not applicable; e.g., when one does not know the exact probability of each message.
- Universal codes are also useful when Huffman coding is inconvenient. For example, when the transmitter but not the receiver knows the probabilities of the messages, Huffman coding requires an overhead of transmitting those probabilities to the receiver. Using a universal code does not have that overhead.

Lempel-Ziv Coding

There are many variations of Lempel Ziv around. We will only concentrate on one of the simplest to explain and analyze the idea: **parse the sequence into distinct phrases**.

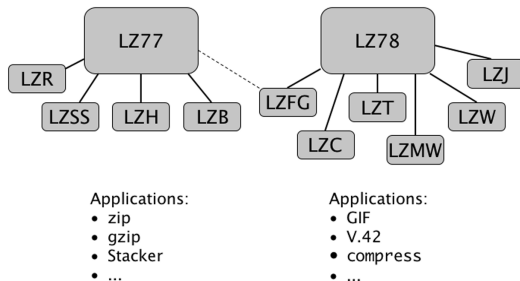


Figure: LZ77 and LZ78 (aka LZ1 and LZ2) are the two lossless data compression algorithms published in papers by Abraham Lempel and Jacob Ziv in 1977 and 1978. Figure source: “The Lempel Ziv Algorithm” by Christina Zeeh, Jan 2013.

Lempel-Ziv Coding: Example (1/2)

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Consider the source sequence as *AABABBBABAABABBBABBABB*.

- Start with the shortest phrase that we haven't seen before. This will always be a single letter, in this case A:

$$A|AABABBBABAABABBBABBABB$$

- Now take the next phrase we haven't seen. So we take AB:

$$A|AB|AABABBBABAABABBBABBABB$$

- The next phrase we haven't seen is ABB. Continuing, we get B after that:

$$A|AB|ABB|B|ABAABABBBABBABB$$

- The rest of the string parses into:

$$A|AB|ABB|B|ABA|ABAB|BB|ABBA|BB$$

It's fine to have a repeated phrase at the end since we've run out of letters.

Lempel-Ziv Coding: Example (2/2)

Index	1	2	3	4	5	6	7	8	9
Phrase	A	AB	ABB	B	ABA	ABAB	BB	ABBA	BB
Encoding	$\emptyset$ A	1B	2B	$\emptyset$ B	2A	5B	4B	3A	7
Codeword	0	11	101	001	0100	1011	1001	0110	0111

Table: LZ coding scheme.

- Each phrase is broken up into a reference (in blue) to a previous phrase, as well as a letter (in red) that is always an innovation. To get the final LZ code, we convert both parts into binary codes with a mapping $A \rightarrow 0$ and $B \rightarrow 1$.
- Start with the $2^{k-1} + 1$ dictionary element (i.e., index = 2, 3, 5, 9, ... for $k = 1, 2, 3, 4, \dots$), the reference part uses k bits. So the number of bits used for this part keeps increasing. This ensures that the decoding algorithm knows where to divide.

Comparison of Source Coding Schemes

Taxonomy of
Data
Compression

Huffman coding

Shannon-Fano
coding

Shannon-Fano-
Elias coding

Shannon coding

Arithmetic
coding

Lempel-Ziv
coding

Code	Optimal	Practical	Needs PMF	Achieves H
Huffman	Optimal (symbol)	Yes	Yes	$\leq H + 1$
Shannon	Suboptimal	Yes	Yes	$\leq H + 1$
SFE	Worse	No	Yes	$\leq H + 2$
Arithmetic	Asymp. optimal	Yes	Yes	$\rightarrow H$
LZ	Universal	Yes	No	$\rightarrow H$